



作业习题评讲课

数据库系统原理课程

2025

第一次作业 2.1

employee (*person_name*, *street*, *city*)
works (*person_name*, *company_name*, *salary*)
company (*company_name*, *city*)

Figure 2.17 Employee database.

2.1 Consider the employee database of Figure 2.17. What are the appropriate primary keys?

employee 和 *works* 表中选择 *person_name* 为主键,
company 表中选择 *company_name* 为主键。

第一次作业 2.6

employee (*person_name*, *street*, *city*)
works (*person_name*, *company_name*, *salary*)
company (*company_name*, *city*)

Figure 2.17 Employee database.

2.6 Consider the employee database of Figure 2.17. Give an expression in the relational algebra to express each of the following queries:

- a. Find the name of each employee who lives in city “Miami”.
- b. Find the name of each employee whose salary is greater than \$100000.
- c. Find the name of each employee who lives in “Miami” and whose salary is greater than \$100000.

答案:

- a. $\Pi_{person_name}(\sigma_{city="Miami"}(employee))$
- b. $\Pi_{person_name}(\sigma_{salary>100000}(works))$
- c. $\Pi_{person_name}(\sigma_{salary>100000 \wedge city="Miami"}(employee \bowtie works))$

第一次作业 2.7

branch(*branch_name*, *branch_city*, *assets*)
customer (*ID*, *customer_name*, *customer_street*, *customer_city*)
loan (*loan_number*, *branch_name*, *amount*)
borrower (*ID*, *loan_number*)
account (*account_number*, *branch_name*, *balance*)
depositor (*ID*, *account_number*)

Figure 2.18 Bank database.

- 2.7 Consider the bank database of Figure 2.18. Give an expression in the relational algebra for each of the following queries:
- Find the name of each branch located in “Chicago”.
 - Find the ID of each borrower who has a loan in branch “Downtown”.

答案:

- $\Pi_{branch_name}(\sigma_{branch_city="Chicago"}(branch))$
- $\Pi_{ID}(\sigma_{branch_name="Downtown"}(borrower \bowtie loan))$

第一次作业 2.8

employee (*person_name*, *street*, *city*)
works (*person_name*, *company_name*, *salary*)
company (*company_name*, *city*)

Figure 2.17 Employee database.

2.8 Consider the employee database of Figure 2.17. Give an expression in the relational algebra to express each of the following queries:

- a. Find the ~~ID~~ and name of each employee who does not work for “BigBank”.
- b. Find the ~~ID~~ and name of each employee who earns at least as much as every employee in the database.

答案:

- a. $\Pi_{person_name}(works) - \Pi_{person_name}(\sigma_{company_name="BigBank"}(works))$
- b. $\Pi_{person_name}(works) - \Pi_{person_name}(\sigma_{works.salary < d.salary}(works \times \rho_d(\Pi_{salary}(works))))$

第一次作业 2.15

```
branch(branch_name, branch_city, assets)
customer (ID, customer_name, customer_street, customer_city)
loan (loan_number, branch_name, amount)
borrower (ID, loan_number)
account (account_number, branch_name, balance)
depositor (ID, account_number)
```

Figure 2.18 Bank database.

2.15 Consider the bank database of Figure 2.18. Give an expression in the relational algebra for each of the following queries:

- a. Find each loan number with a loan amount greater than \$10000.
- b. Find the ID of each depositor who has an account with a balance greater than \$6000.
- c. Find the ID of each depositor who has an account with a balance greater than \$6000 at the "Uptown" branch.

答案:

- a. $\Pi_{loan_number}(\sigma_{amount > 10000}(loan))$
- b. $\Pi_{ID}(\sigma_{balance > 6000}(depositor \bowtie account))$
- c. $\Pi_{ID}(\sigma_{balance > 6000 \wedge branch_name = "Uptown"}(depositor \bowtie account))$

注意区分是branch的name还是所处的city

第二次作业 3.8

3.8 Consider the bank database of Figure 3.18, where the primary keys are under-lined. Construct the following SQL queries for this relational database.

branch(*branch_name*, *branch_city*, *assets*)
customer (*ID*, *customer_name*, *customer_street*, *customer_city*)
loan (*loan_number*, *branch_name*, *amount*)
borrower (*ID*, *loan_number*)
account (*account_number*, *branch_name*, *balance*)
depositor (*ID*, *account_number*)

Figure 3.18 Banking database.

a. Find the ID of each customer of the bank who has an account but not a loan.

(select ID
from depositor)
except
(select ID
from borrower)

第二次作业 3.8

b. Find the ID of each customer who lives on the same street and in the same city as customer '12345'.

select F.ID

from customer as F, customer as S

where F.customer_street = S.customer_street

and F.customer_city = S.customer_city

and S.customer_id = '12345'

branch(branch_name, branch_city, assets)

customer (ID, customer_name, customer_street, customer_city)

loan (loan_number, branch_name, amount)

borrower (ID, loan_number)

account (account_number, branch_name, balance)

depositor (ID, account_number)

c. Find the name of each branch that has at least one customer who has an account in the bank and who lives in "Harrison".

select distinct branch_name

from account, depositor, customer

where customer.id = depositor.id

and depositor.account_number = account.account_number

and customer_city = 'Harrison'

branch(branch_name, branch_city, assets)

customer (ID, customer_name, customer_street, customer_city)

loan (loan_number, branch_name, amount)

borrower (ID, loan_number)

account (account_number, branch_name, balance)

depositor (ID, account_number)

第二次作业 3.9

3.9 Consider the relational database of Figure 3.19, where the primary keys are underlined. Give an expression in SQL for each of the following queries.

- Find the ID, name, and city of residence of each employee who works for "First Bank Corporation".
- Find the ID, name, and city of residence of each employee who works for "First Bank Corporation" and earns more than \$10000.
- Find the ID of each employee who does not work for "First Bank Corporation".
- Find the ID of each employee who earns more than every employee of "Small Bank Corporation".
- Assume that companies may be located in several cities. Find the name of each company that is located in every city in which "Small Bank Corporation" is located.
- Find the name of the company that has the most employees (or companies, in the case where there is a tie for the most).
- Find the name of each company whose employees earn a higher salary, on average, than the average salary at "First Bank Corporation".

employee (ID, person_name, street, city)
works (ID, company_name, salary)
company (company_name, city)
manages (ID, manager_id)

- select E.ID, E.person_name, E.city*
from employee as E, works as W
where E.ID = W.ID and W.company_name = "First Bank Corporation"
- select E.ID, E.person_name, E.city*
from employee as E, works as W
where E.ID = W.ID
and W.company_name = "First Bank Corporation"
and W.salary > 10000
- select ID*
from works
where company_name < > "First Bank Corporation"
- select ID*
from works
where salary > all(select salary
from works
where company_name = "Small Bank Corporation"

第二次作业 3.9

- e. Assume that companies may be located in several cities. Find the name of each company that is located in every city in which "Small Bank Corporation" is located.
- f. Find the name of the company that has the most employees (or companies, in the case where there is a tie for the most).
- g. Find the name of each company whose employees earn a higher salary, on average, than the average salary at "First Bank Corporation".

employee (ID, *person_name*, *street*, *city*)
works (ID, *company_name*, *salary*)
company (*company_name*, *city*)
manages (ID, *manager_id*)

- e. *select C.company_name*
from company as C
where not exists //若集合差为空, not exist 返回 True, 即该公司满足位于Small Bank Corporation 所在的所有城市
(select city //SBC所在的所有城市
from company
where company_name = "Small Bank Corporation"
except (select D.city from company as D //C对应的公司所在的所有城市
where D.company_name = C.company_name))
- f. *select company_name*
from works
group by company_name
having count(distinct ID) >= all(
select count(distinct ID) from works group by company_name)
- g. *select company_name*
from works
group by company_name
having avg(salary) > (select avg(salary) from works
where company_name = "First Bank Corporation")

第二次作业 3.15

3.15 Consider the bank database of Figure 3.18, where the primary keys are underlined. Construct the following SQL queries for this relational database.

branch(*branch_name*, *branch_city*, *assets*)
customer (*ID*, *customer_name*, *customer_street*, *customer_city*)
loan (*loan_number*, *branch_name*, *amount*)
borrower (*ID*, *loan_number*)
account (*account_number*, *branch_name*, *balance*)
depositor (*ID*, *account_number*)

- Find each customer who has an account at every branch located in "Brooklyn".
- Find the total sum of all loan amounts in the bank.
- Find the names of all branches that have assets greater than those of at least one branch located in "Brooklyn".

第二次作业 3.15

branch(branch_name, branch_city, assets)
customer (ID, customer_name, customer_street, customer_city)
loan (loan_number, branch_name, amount)
borrower (ID, loan_number)
account (account_number, branch_name, balance)
depositor (ID, account_number)

- a. Find each customer who has an account at every branch located in "Brooklyn".
- b. Find the total sum of all loan amounts in the bank.
- c. Find the names of all branches that have assets greater than those of at least one branch located in "Brooklyn".

a. *select distinct C.customer_name*
from customer as C
where not exists (

(select branch_name 找出位于 ‘Brooklyn’的所有branch的name
from branch
where branch_city = 'Brooklyn')

except 找出C的所有存款所在的branch的name
(select branch_name
from account join depositor using(account_number)
where C.ID = depositor.ID))

b. *select sum(amount)*
from loan

c. *select branch_name*
from branch
where assets > some
(select assets
from branch
where branch_city = "Brooklyn")

第二次作业 3.16

3.16 Consider the employee database of Figure 3.19, where the primary keys are underlined. Give an expression in SQL for each of the following queries.

- Find ID and name of each employee who lives in the same city as the location of the company for which the employee works.
- Find ID and name of each employee who lives in the same city and on the same street as does her or his manager.
- Find ID and name of each employee who earns more than the average salary of all employees of her or his company.
- Find the company that has the smallest payroll.

employee (ID, person_name, street, city)
works (ID, company_name, salary)
company (company_name, city)
manages (ID, manager_id)

Figure 3.19 Employee database.

a. *select ID, person_name*
from (*employee natural join works*) *natural join* *company*
where employee.ID=works.ID and works.company_name
=company.company_name and employee.city=company.city

c. *select ID, person_name*
from works as A
where salary > (select avg(salary)
from works as W
where W. company_name = A. company_name)

d. *select company_name*
from works
group by company_name
having sum(salary) <= all(select sum(salary)
from works
group by company_name)

b. *select E.ID, E.person_name*
from employee as E, employee as D, manages
where E.ID = manages.ID
and D.ID = manages.manager_id
and E.street = D.street
and E.city = D.city

第二次作业 3.17

3.17 Consider the employee database of Figure 3.19. Give an expression in SQL for each of the following queries.

- Give all employees of “First Bank Corporation” a 10 percent raise.
- Give all managers of “First Bank Corporation” a 10 percent raise.
- Delete all tuples in the *works* relation for employees of “Small Bank Corporation”.

employee (ID, person_name, street, city)
works (ID, company_name, salary)
company (company_name, city)
manages (ID, manager_id)

Figure 3.19 Employee database.

- update*** *works*
set salary = salary * 1.1
where company_name = "First Bank Corporation"
- update*** *works*
set salary = salary * 1.1
where works.company_name = "First Bank Corporation"
and ID in (***select*** distinct manager_id
from manages)
- delete from*** *works*
where company_name = "Small Bank Corporation"

第二次作业 3.21

3.21 Consider the library database of Figure 3.20. Write the following queries in SQL.

- Find the member number and name of each member who has borrowed at least one book published by “McGraw-Hill”.
- Find the member number and name of each member who has borrowed every book published by “McGraw-Hill”.
- For each publisher, find the member number and name of each member who has borrowed more than five books of that publisher.
- Find the average number of books borrowed per member. Take into account that if a member does not borrow any books, then that member does not appear in the *borrowed* relation at all, but that member still counts in the average.

member(*memb_no*, *name*)
book(*isbn*, *title*, *authors*, *publisher*)
borrowed(*memb_no*, *isbn*, *date*)

a. ***select M.memb_no, M.name***
from member M, book B, borrowed C
where C.memb_no = M.memb_no
and B.isbn = C.isbn
and B.publisher = "McGraw-Hill"

b. ***select M.memb_no, M.name***
from member M
where not exist (select isbn from book
where B.publisher = "McGraw-Hill")
except (select isbn from borrowed E
where E.memb_no = M.memb_no)

c. ***select B.publisher, M.memb_no, M.name***
from member M, book B, borrowed C
where C.memb_no = M.memb_no
and B.isbn = C.isbn
group by B.publisher, M.memb_no, M.name
having count(*) > 5

d. ***Select avg(borrowed_count) as avg_borrowed_count***
from (select ,count(b.isbn) as borrowed_count
from member as M
left join borrowed as B on M.memb_no = B.memb_no
group by M.memb_no
)as member_borrowed_count

第三次作业 4.7

- Consider the employee database of Figure 4.12. Give an SQL DDL definition of this database. Identify referential-integrity constraints that should hold, and include them in the DDL definition.

employee (ID, person_name, street, city)
works (ID, company_name, salary)
company (company_name, city)
manages (ID, manager_id)

Figure 4.12 Employee database.

第三次作业 4.7

employee (ID, person_name, street, city)
works (ID, company_name, salary)
company (company_name, city)
manages (ID, manager_id)

Figure 4.12 Employee database.

- 参考完整性约束:

- works表中:

- employee_name** 应该是关联到 **employee** 表的主键 **employee_name** 的外键

- company_name** 应该是关联到 **company** 表的主键 **company_name** 的外键

- manages表中:

- employee_name** 应该是关联到 **employee** 表的主键 **employee_name** 的外键

- manager_name** 应该是关联到 **employee** 表的主键 **employee_name** 的外键

第三次作业 4.7

- DDL定义:

CREATE TABLE employee

```
(  
    employee_name VARCHAR (20),  
    street VARCHAR (50),  
    city VARCHAR (50),  
    PRIMARY KEY (employee_name)  
);
```

CREATE TABLE works

```
(  
    employee_name VARCHAR (20),  
    company_name VARCHAR (30),  
    salary NUMERIC (10,2),  
    PRIMARY KEY (employee_name),  
    FOREIGN KEY (employee_name) REFERENCES employee,  
    FOREIGN KEY (company_name) REFERENCES company  
);
```

CREATE TABLE company

```
(  
    company_name VARCHAR (30),  
    city VARCHAR (50),  
    PRIMARY KEY (company_name)  
);
```

CREATE TABLE manages

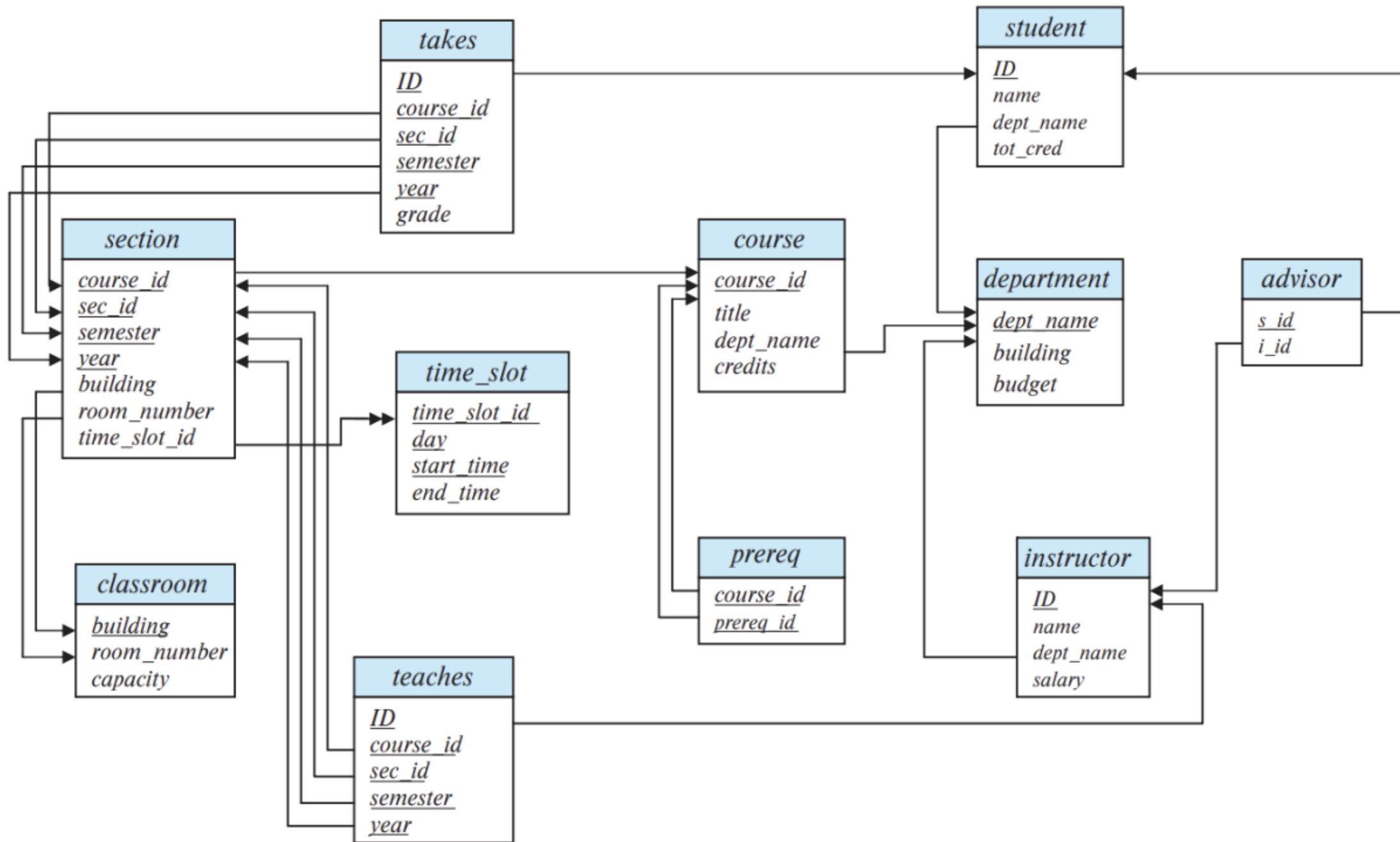
```
(  
    employee_name VARCHAR (20),  
    manager_name VARCHAR (20),  
    PRIMARY KEY (employee_name),  
    FOREIGN KEY (employee_name) REFERENCES employee,  
    FOREIGN KEY (manager_name) REFERENCES employee  
);
```

第三次作业 4.16

- Write an SQL query using the university schema to find the ID of each student who has never taken a course at the university. Do this using no subqueries and no set operations (use an outer join).

classroom(building, room_number, capacity)
department(dept_name, building, budget)
course(course_id, title, dept_name, credits)
instructor(ID, name, dept_name, salary)
section(course_id, sec_id, semester, year, building, room_number, time_slot_id)
teaches(ID, course_id, sec_id, semester, year)
student(ID, name, dept_name, tot_cred)
takes(ID, course_id, sec_id, semester, year, grade)
advisor(s_ID, i_ID)
time_slot(time_slot_id, day, start_time, end_time)
prereq(course_id, prereq_id)

第三次作业 4.16



```
SELECT s.ID
FROM student s LEFT OUTER JOIN takes t
ON s.ID = t.ID
WHERE t.ID IS NULL;
```

假设 `student` 表和 `takes` 表中有以下数据：

`student` 表 (左表)

ID	name	dept_name
12345	张三	计算机
54321	王五	物理
67890	李四	音乐

`takes` 表 (右表)

ID	course_id	semester	year
12345	CS-101	Fall	2023
54321	PHY-101	Fall	2023
12345	CS-347	Spring	2024

执行 `FROM student s LEFT OUTER JOIN takes t ON s.ID = t.ID` 后，生成的临时结果表会是下面这个样子：

s.ID	s.name	s.dept_name	t.ID	t.course_id	t.semester	t.year
12345	张三	计算机	12345	CS-101	Fall	2023
12345	张三	计算机	12345	CS-347	Spring	2024
54321	王五	物理	54321	PHY-101	Fall	2023
67890	李四	音乐	NULL	NULL	NULL	NULL

第四次作业 7.1

Suppose that we decompose the schema $R = (A, B, C, D, E)$ into

$$(A, B, C) \\ (A, D, E).$$

Show that this decomposition is a lossless decomposition if the following set F of functional dependencies holds:

$$A \rightarrow BC \\ CD \rightarrow E \\ B \rightarrow D \\ E \rightarrow A$$

定义法
列表法

$$R_1 = (A, B, C) \cdot R_2 = (A, D, E) \leftarrow$$

$$R_1 \cap R_2 = A \cdot \text{and} \cdot A \rightarrow ABC \leftarrow$$

所以是无损分解 $\leftarrow$

第四次作业 7.6

Compute the closure of the following set F of functional dependencies for relation schema $R = (A, B, C, D, E)$.

$$\begin{aligned}A &\rightarrow BC \\ CD &\rightarrow E \\ B &\rightarrow D \\ E &\rightarrow A\end{aligned}$$

List the candidate keys for R .

因为有 $A \rightarrow BC$, 所以可以得到 $A \rightarrow B$ 与 $A \rightarrow C$

因为 $A \rightarrow B$ 与 $B \rightarrow D$, 所以有 $A \rightarrow D$

因为 $A \rightarrow CD$ 与 $CD \rightarrow E$, 所以有 $A \rightarrow E$

综上所述 $A \rightarrow ABCDE$

因为 $E \rightarrow A$, 所以 $E \rightarrow ABCDE$

因为 $CD \rightarrow E$, 所以 $CD \rightarrow ABCDE$

因为 $B \rightarrow D$ 且 $BC \rightarrow CD$, 所以有 $BC \rightarrow ABCDE$

综上所述, 候选码有: A, E, CD, BC

参考答案的做法不泛用

第四次作业 7.6

寻找候选码 (续)

- **算法**：寻找关系模式 H 的候选码
- **输入**：关系模式 H 及其函数依赖集 F
- **输出**： H 的所有候选码

- (1) 将所有属性分为两类： X 代表 L 型（只在函数依赖左部出现）和 N 型（从未出现）的属性， Y 代表 LR 型（在左右两边都出现）的属性。
- (2) 计算 X 的属性闭包 X^+ 。如果 X^+ 包含了 H 的所有属性，那么 X 是 H 的唯一候选码，转到步骤 (5)；否则，转到步骤 (3)。
- (3) 从 Y 中取出一个属性 A ，计算 $(XA)^+$ 。如果 $(XA)^+$ 包含了 H 的所有属性，那么 XA 就是 H 的一个候选码。然后从 Y 中取另一个属性，重复此过程，直到 Y 中所有属性都被测试完毕。
- (4) 如果在步骤 (3) 中已经找到了所有候选码，则转到步骤 (5)；否则，从 Y 中取2个、3个或更多属性组成属性组，计算其对应的属性闭包（注意：该属性组不应包含任何在之前步骤中已找到的候选码），直到其闭包包含 H 的所有属性。
- (5) 结束，并输出结果。

步骤 (1): 属性分类

首先, 我们需要将所有属性 $\{A, B, C, D, E\}$ 分类。算法中提到了 L, N, Y (LR) 三种类型, 我们发现, 所有属性 A, B, C, D, E 都在左右两侧出现过。因此:

- L 型属性: 无
- N 型属性: 无
- LR 型属性: $\{A, B, C, D, E\}$

根据算法, 我们将 L 型和 N 型属性放入集合 X , 将 LR 型放入集合 Y 。

- $X = \emptyset$ (空集)
- $Y = \{A, B, C, D, E\}$

步骤 (2): 计算 X 的闭包

计算 X^+ 。因为 X 是空集, 所以 $X^+ = \emptyset^+ = \emptyset$ 。

这个闭包不包含 R 的所有属性, 所以我们进入步骤 (3)。

步骤 (3): 逐个测试 Y 中的属性

我们从集合 Y 中逐个取出属性 A , 并计算 $(X \cup \{A\})^+$, 即 A^+ 。

- $\{A\}^+ = \{A\} \rightarrow$ (根据 $A \rightarrow BC$) $\rightarrow \{A, B, C\} \rightarrow$ (根据 $B \rightarrow D$) $\rightarrow \{A, B, C, D\} \rightarrow$ (根据 $CD \rightarrow E$) $\rightarrow \{A, B, C, D, E\}$ 。
- 结果包含了 R 的所有属性, 因此 **A** 是一个候选码。同理**E** 是一个候选码。

步骤 (4): 测试 Y 中属性的组合

我们在步骤 (3) 中已经找到了候选码 **A** 和 **E**。现在需要测试由2个或更多属性组成的组合。根据算法提示, 要测试的属性组不应包含已经找到的候选码。所以我们从剩余的属性 $\{B, C, D\}$ 中进行组合。

两两组合, **BC** 是一个候选码。同理, **CD** 是一个候选码。

- 其他组合 (如 **BD**) 的闭包不包含所有属性, 而包含3个属性的组合 (如 **BCD**) 必然不是最小的, 因为它的子集 **BC** 和 **CD** 已经是候选码了。

步骤 (5): 完成并输出结果

算法执行完毕。

最终候选码列表为: **A, E, BC, CD**。

第四次作业 7.30

Consider the following set F of functional dependencies on the relation schema (A, B, C, D, E, G) :

$$A \rightarrow BCD$$

$$BC \rightarrow DE$$

$$B \rightarrow D$$

$$D \rightarrow A$$

a. Compute B^+ .

因为 $B \rightarrow D$ 且 $D \rightarrow A$ 所以有 $B \rightarrow ABD$

因为 $A \rightarrow BCD$, 所以有 $B \rightarrow ABCD$

因为 $BC \rightarrow DE$, 所以有 $B \rightarrow ABCDE$

综上所述, $B^+ = \{A, B, C, D, E\}$

第四次作业 7.30

b. Prove (using Armstrong's axioms) that AG is a superkey

$A \rightarrow BCD$
 $BC \rightarrow DE$
 $B \rightarrow D$
 $D \rightarrow A$

$A \rightarrow BCD$, 由分解律: $A \rightarrow BC$

因为 $BC \rightarrow DE$, 由传递律: $A \rightarrow DE$

由合并律: $A \rightarrow BCDE$

由增广律: $AG \rightarrow ABCDEG$

所以 AG 是超码

验证超码: 计算其属性闭包是否包含所有属性

第四次作业 7.30

c. Compute a canonical cover for this set of functional dependencies F ; give each step of your derivation with an explanation.

- 对于 $A \rightarrow BCD$, D 是无关属性, 因为在 $(F - \{A \rightarrow BCD\}) \cup (A \rightarrow BC)$ 中, $A \rightarrow BC$ 和 $B \rightarrow D$ 可以推出 $A \rightarrow D$
- 对于 $BC \rightarrow DE$, D 是无关属性, 因为在 $(F - \{BC \rightarrow DE\}) \cup (BC \rightarrow E)$ 中, $B \rightarrow D$ 可以推出 $BC \rightarrow D$
- 对于 $BC \rightarrow E$, C 是无关属性, 因为由 $B \rightarrow D$ 、 $D \rightarrow A$ 、 $A \rightarrow BC$ 可以推出 $B \rightarrow C$, 而由增广律可以得到 $B \rightarrow BC$, 再由合并律即可得到 $B \rightarrow E$, 因此 C 是无关属性
- 此时 F 可被简化为 $\{A \rightarrow BC, B \rightarrow E, B \rightarrow D, D \rightarrow A\}$
- 由于有两个左侧为 B 的函数依赖, 则将其合并, 可以得到 F 的正则覆盖为:

$$A \rightarrow BC$$

$$B \rightarrow DE$$

$$D \rightarrow A$$

$$A \rightarrow BCD$$

$$BC \rightarrow DE$$

$$B \rightarrow D$$

$$D \rightarrow A$$

C. $A \rightarrow B$ $A \rightarrow C$ ~~$A \rightarrow D$~~
 ~~$B \rightarrow F$~~ ~~$BC \rightarrow E$~~ $B \rightarrow D$ $D \rightarrow A$
 $B \rightarrow E$

① 对 $B \rightarrow D$ 去掉 C 得 $B \rightarrow D$.

(2) $A \rightarrow B \quad B \rightarrow D \Rightarrow A \rightarrow D$

(3) $B \rightarrow D \quad D \rightarrow A \quad A \rightarrow C \Rightarrow B \rightarrow C \quad \therefore B(\neg E) \Rightarrow B \rightarrow \bar{E}$

$$\therefore F_c = \{A \rightarrow BC, B \rightarrow DE, D \rightarrow A\}$$

第四次作业 7.30

- d. Give a 3NF decomposition of the given schema based on a canonical cover.

正则覆盖中没有多余的函数依赖

所以属性集是正则覆盖中其它函数依赖组成的子集

所以三个函数依赖都有自己的关系

$$R_1(A, B, C)$$

$$R_2(B, D, E)$$

$$R_3(D, A)$$

AG是超码且上述关系没有原关系的超码，所以需要添加

$$R_4(A, G)$$

最终结果为

$$R_1(A, B, C)$$

$$R_2(B, D, E)$$

$$R_3(D, A)$$

$$R_4(A, G)$$

$$R_1 = (A, B, C) \quad F_1 = \{A \rightarrow BC\}$$

$$R_2 = (B, D, E) \quad F_2 = \{B \rightarrow DE\}$$

$$R_3 = (D, A) \quad F_3 = \{D \rightarrow A\}$$

$$R_4 = (A, G) \quad F_4 = \emptyset$$

第五次作业

Q1: Construct a B^+ -tree from an empty tree. Each node can hold four pointers

- The sequential values to be inserted are: 10, 12, 27, 5, 9, 15, 30, 7, 17, 26, 19
- Then delete 10 and 15, respectively
- Please give the B^+ trees after each insertion and each deletion

Q2: Compare B^+ -tree and B-tree and describe their difference

插入 10, 12, 27

[10, 12, 27]

插入 5 (导致分裂)

[12]
/ \
[5, 10] [12, 27]

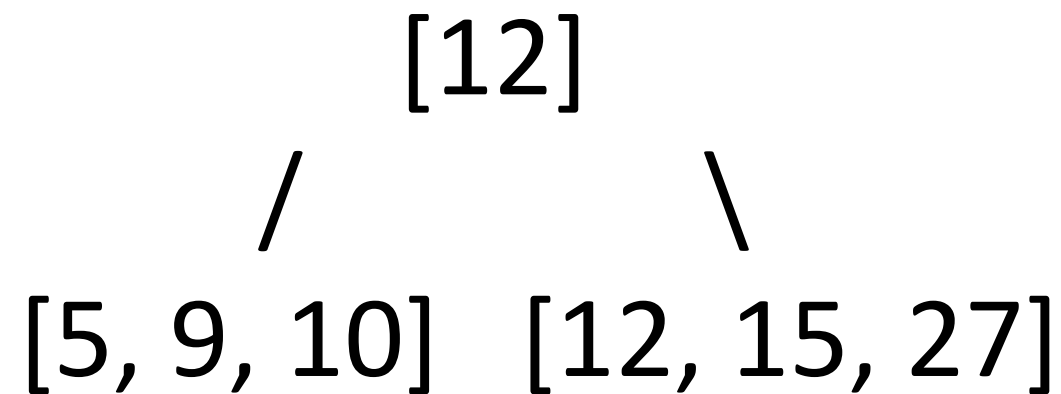
第五次作业

Q1: Construct a B^+ -tree from an empty tree. Each node can hold four pointers

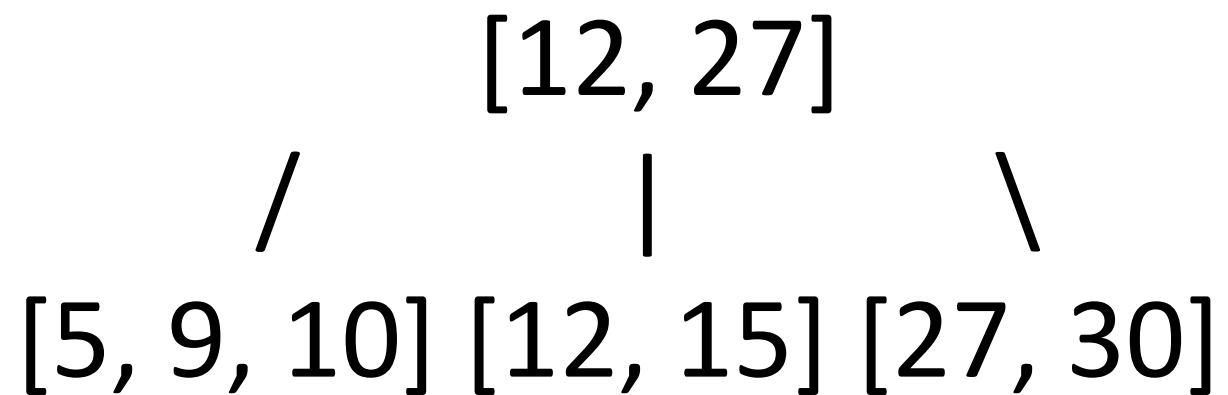
- The sequential values to be inserted are: 10, 12, 27, 5, 9, 15, 30, 7, 17, 26, 19
- Then delete 10 and 15, respectively
- Please give the B^+ trees after each insertion and each deletion

Q2: Compare B^+ -tree and B-tree and describe their difference

插入 9, 15



插入 30 (导致分裂)



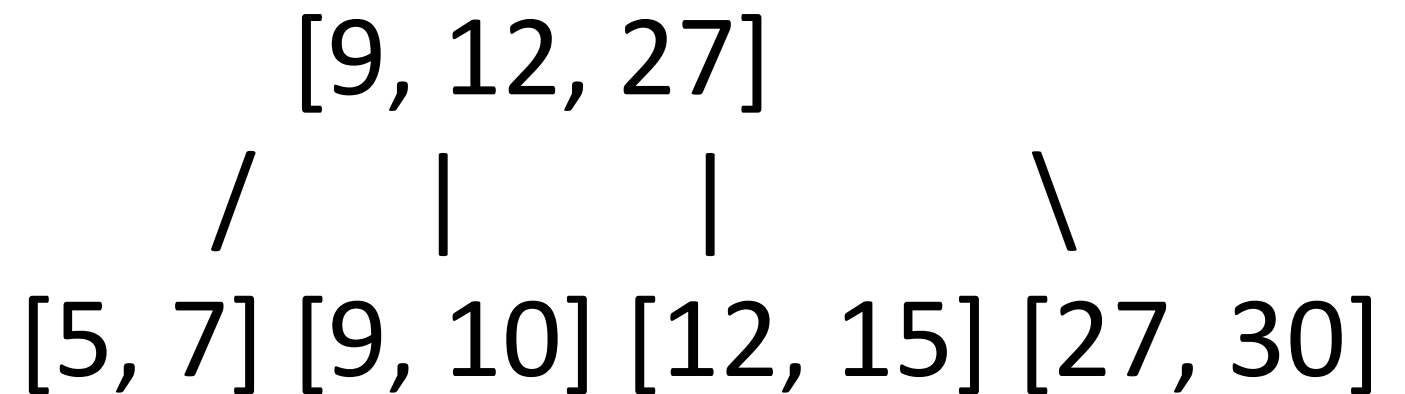
第五次作业

Q1: Construct a B^+ -tree from an empty tree. Each node can hold four pointers

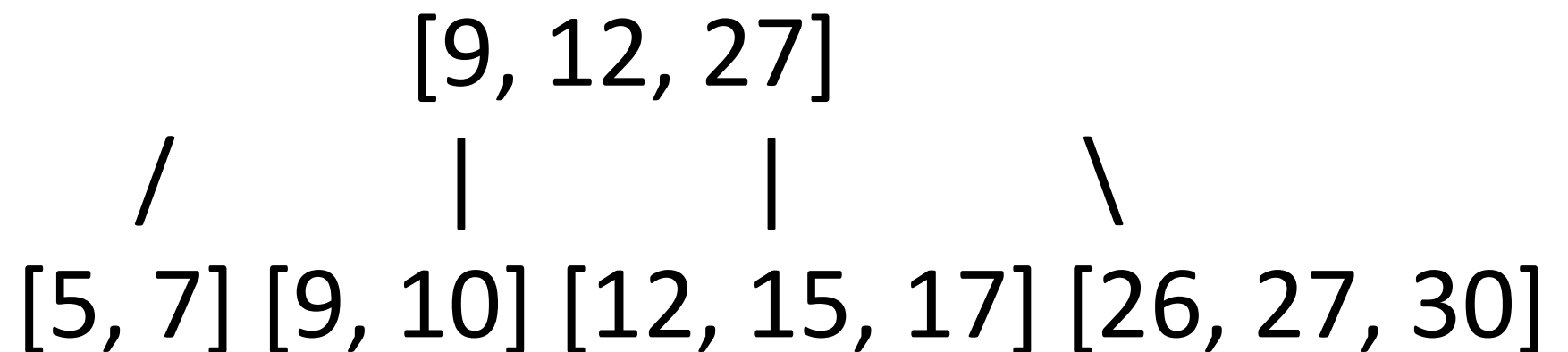
- The sequential values to be inserted are: 10, 12, 27, 5, 9, 15, 30, 7, 17, 26, 19
- Then delete 10 and 15, respectively
- Please give the B^+ trees after each insertion and each deletion

Q2: Compare B^+ -tree and B-tree and describe their difference

插入 7 (导致分裂)



插入 17, 26



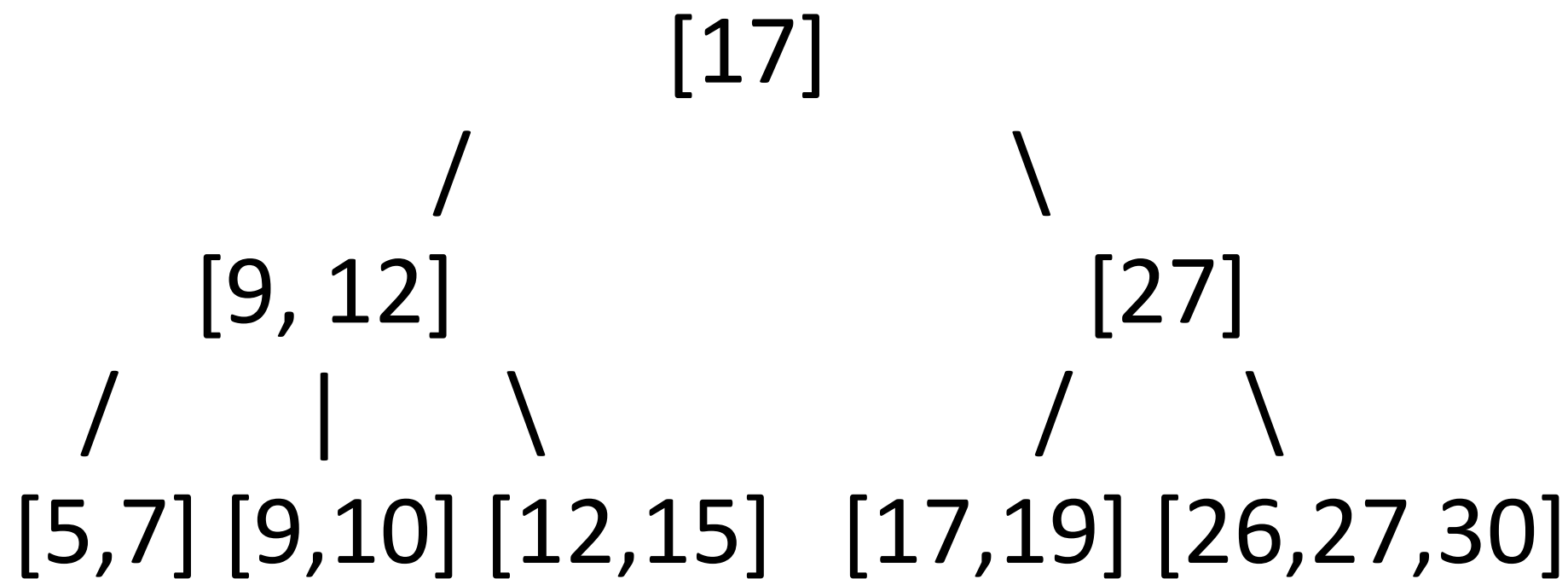
第五次作业

Q1: Construct a B^+ -tree from an empty tree. Each node can hold four pointers

- The sequential values to be inserted are: 10, 12, 27, 5, 9, 15, 30, 7, 17, 26, 19
- Then delete 10 and 15, respectively
- Please give the B^+ trees after each insertion and each deletion

Q2: Compare B^+ -tree and B-tree and describe their difference

插入 19 (导致连续分裂)



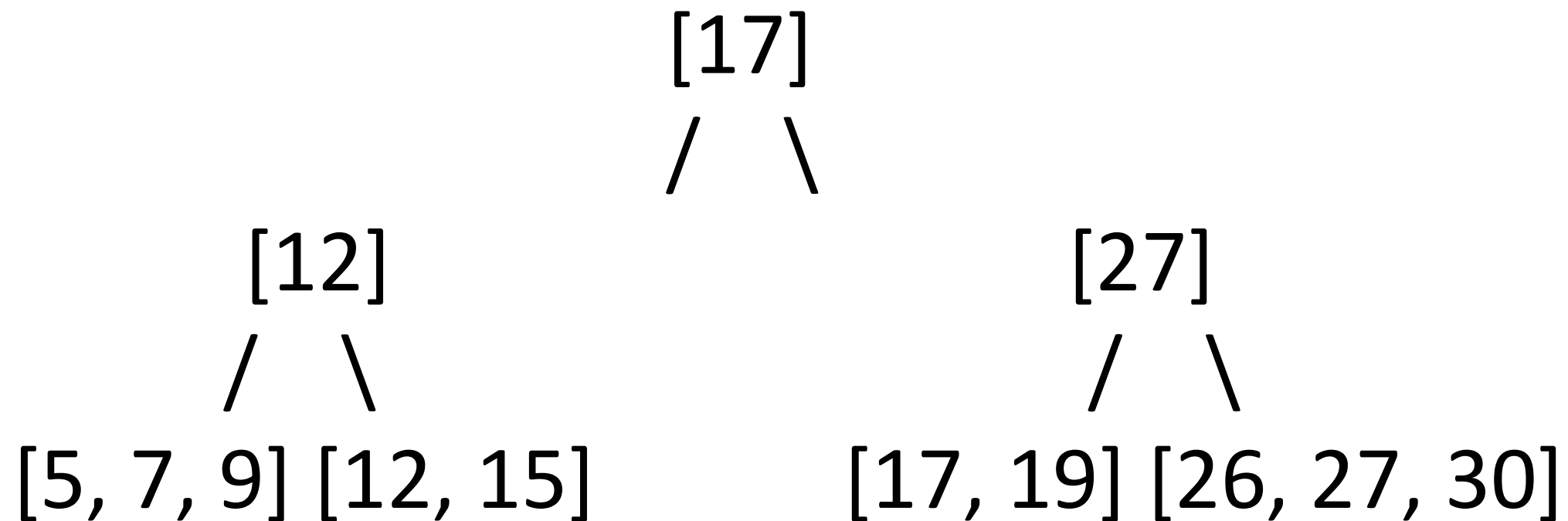
第五次作业

Q1: Construct a B^+ -tree from an empty tree. Each node can hold four pointers

- The sequential values to be inserted are: 10, 12, 27, 5, 9, 15, 30, 7, 17, 26, 19
- Then delete 10 and 15, respectively
- Please give the B^+ trees after each insertion and each deletion

Q2: Compare B^+ -tree and B-tree and describe their difference

删除 10 (导致合并)



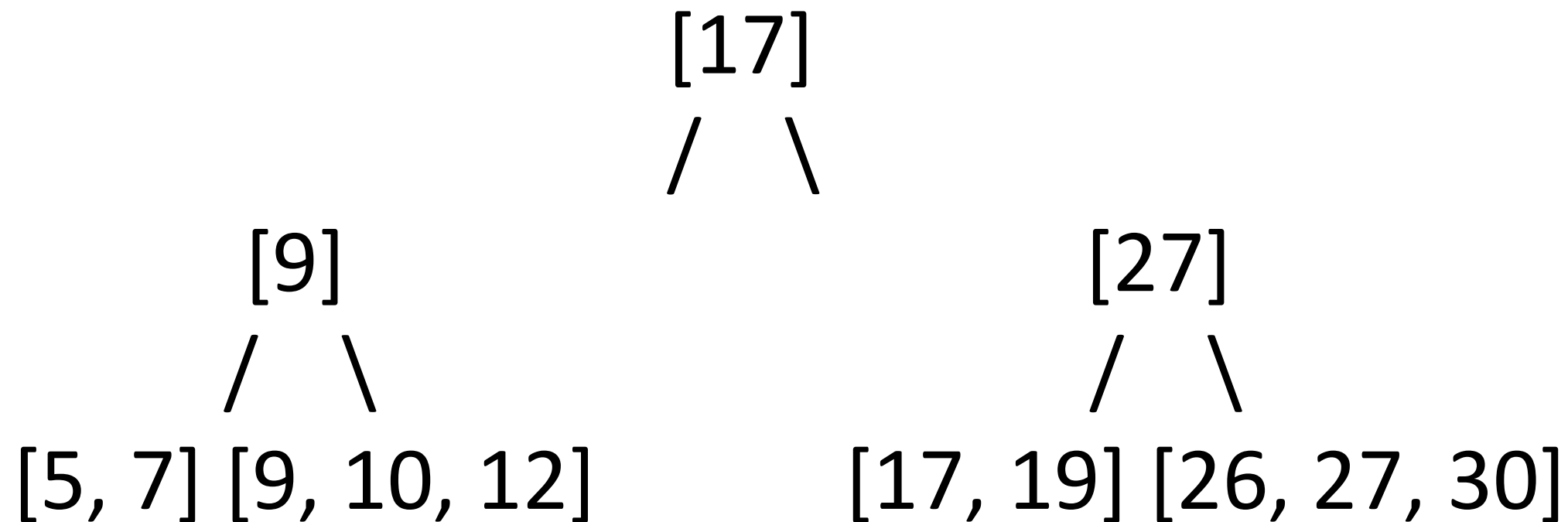
第五次作业

Q1: Construct a B^+ -tree from an empty tree. Each node can hold four pointers

- The sequential values to be inserted are: 10, 12, 27, 5, 9, 15, 30, 7, 17, 26, 19
- Then delete 10 and 15, respectively
- Please give the B^+ trees after each insertion and each deletion

Q2: Compare B^+ -tree and B-tree and describe their difference

删除 15 (导致再平衡)



第五次作业

Q1: Construct a **B⁺-tree** from an empty tree. Each node can hold four pointers

- The sequential values to be inserted are: 10, 12, 27, 5, 9, 15, 30, 7,17, 26, 19
- Then delete 10 and 15, respectively
- Please give the **B⁺ trees** after each insertion and each deletion

Q2: Compare **B⁺-tree** and B-tree and describe their difference

B 树和 B+ 树都是为磁盘等外部存储设备设计的多路平衡查找树，但它们在结构和应用上存在显著差异。B+ 树可以看作是 B 树的一种优化和增强。

特性	B 树 (B-Tree)	B+ 树 (B+-Tree)
数据存储位置	所有节点（内部节点和叶子节点）都存储数据（或指向数据的指针）。	只有 叶子节点 存储数据（或指向数据的指针）。
内部节点结构	内部节点存储 （键，数据） 的组合。	内部节点只存储 键 ，作为索引来引导查找，不存储数据。
叶子节点连接	叶子节点之间 没有 指针连接。	所有叶子节点通过 双向链表 连接起来，形成一个有序序列。
键的冗余	所有键在树中只出现 一次 。	内部节点中的键会 冗余地 出现在叶子节点中。